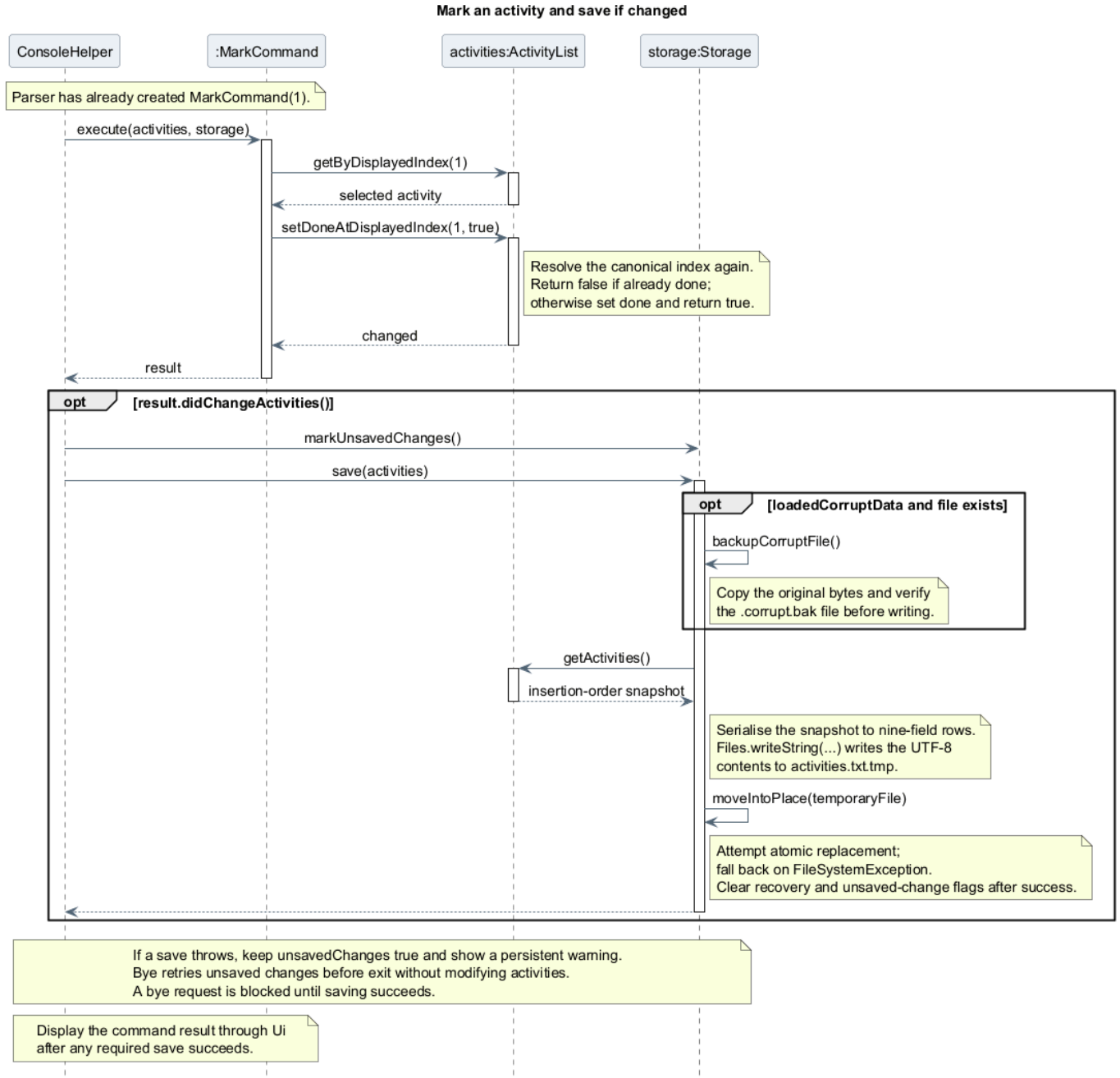
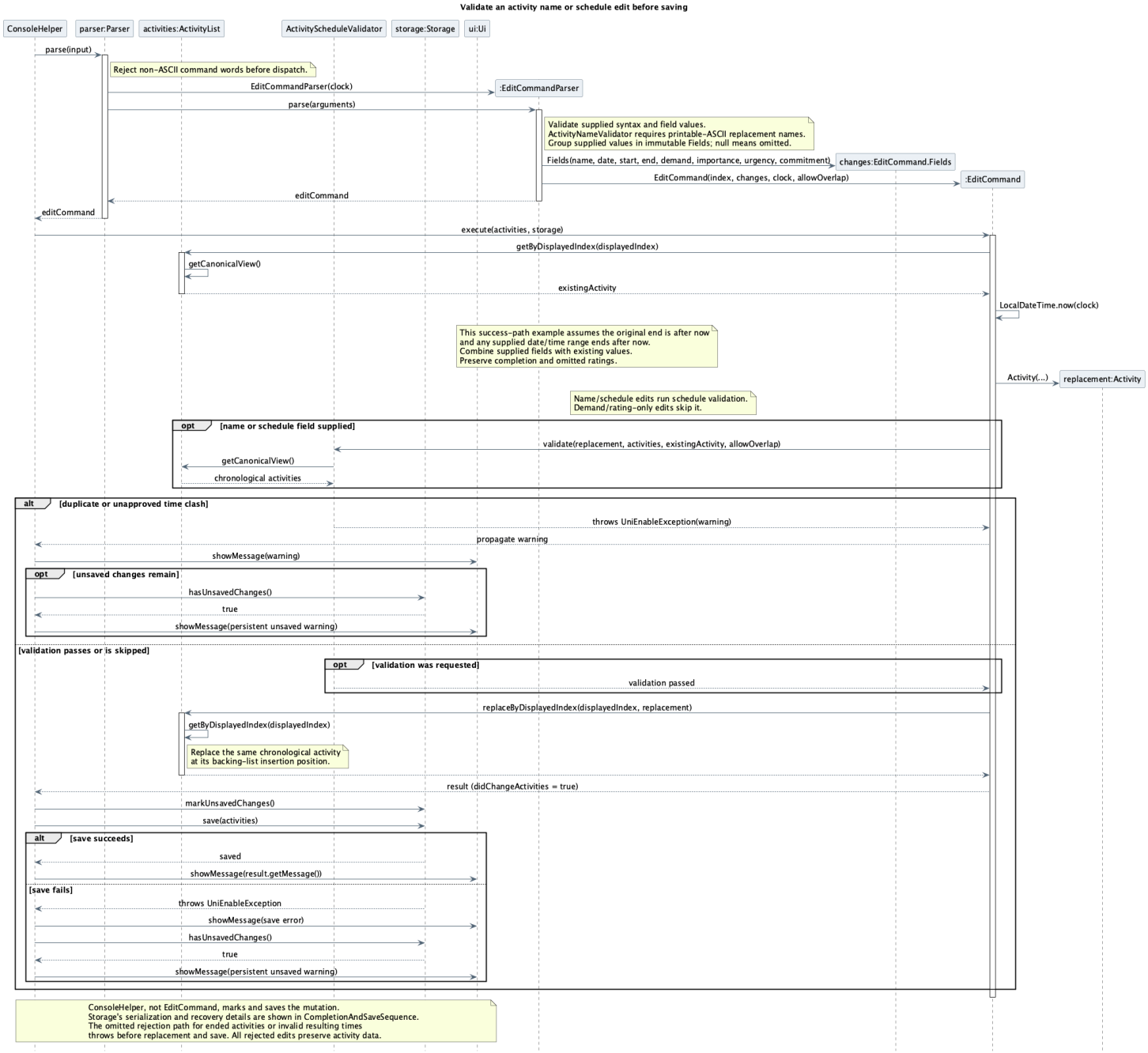
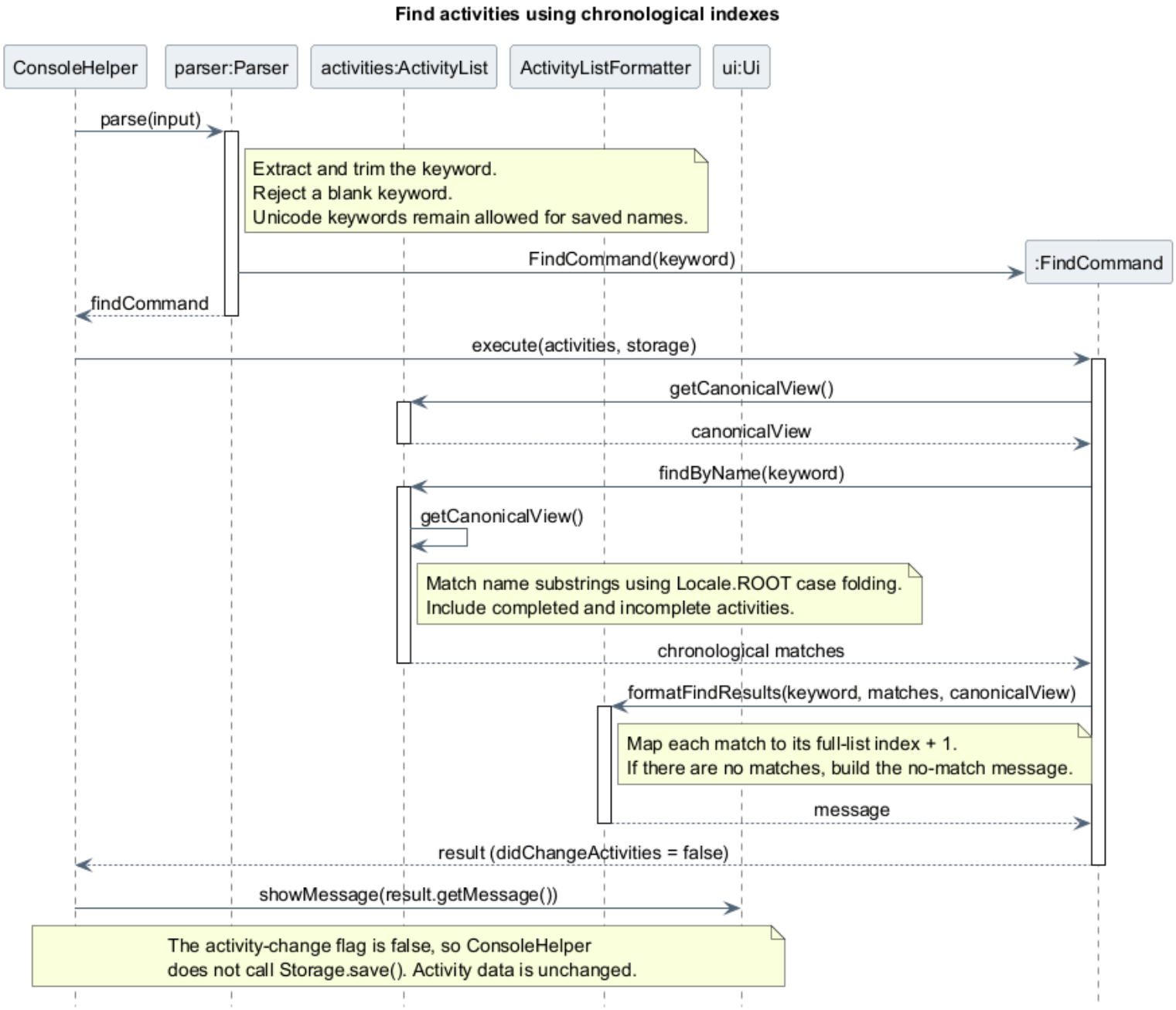
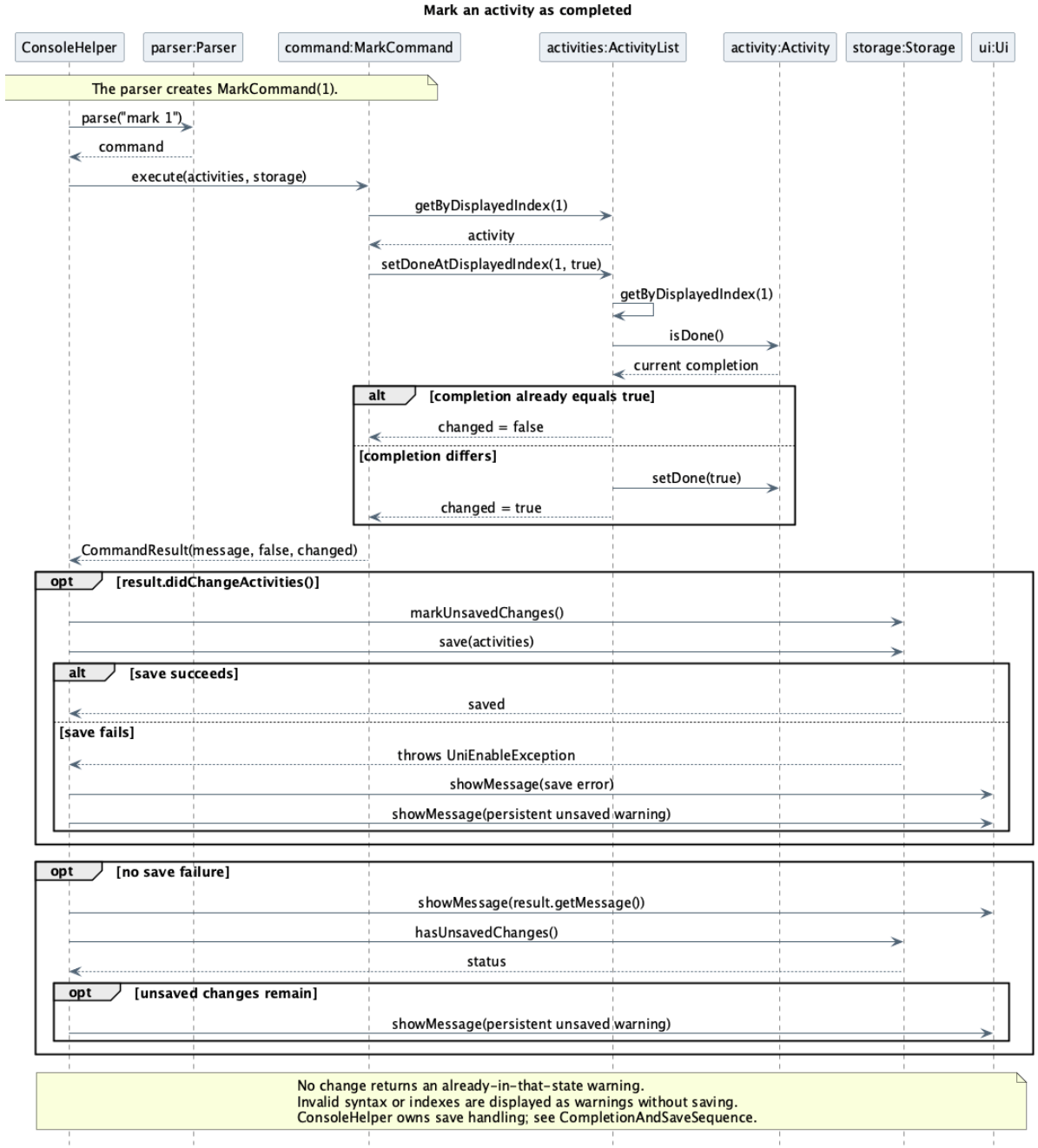
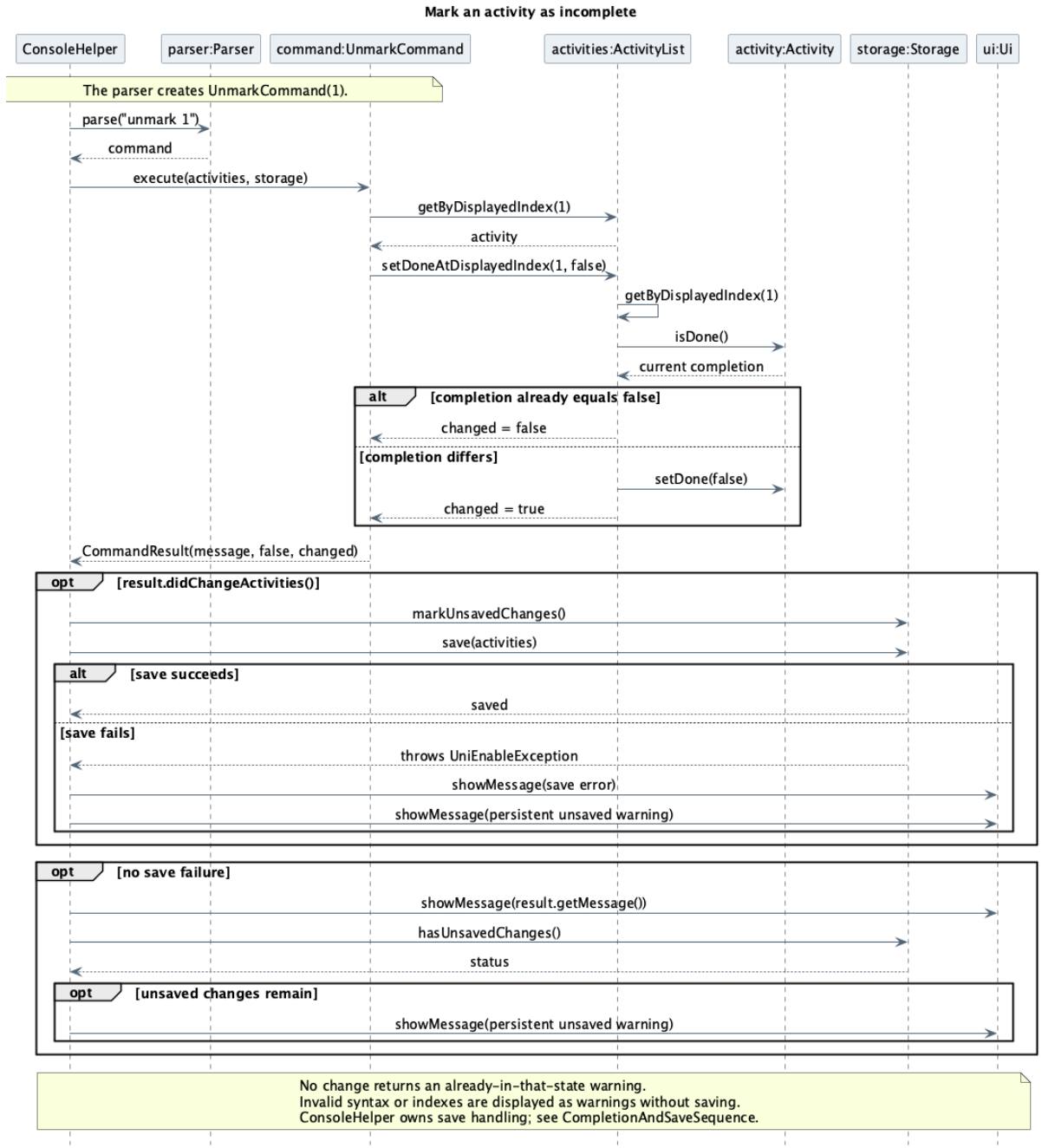

Mark an activity and save if changed
ConsoleHelper
:MarkCommand
activities:ActivityList
storage:Storage
Parser has already created MarkCommand(1).
execute(activities, storage)
getByDisplayedIndex(1)
selected activity
setDoneAtDisplayedIndex(1, true)
Resolve the canonical index again.
Return false if already done;
otherwise set done and return true.
changed
result
opt
[result.didChangeActivities()]
markUnsavedChanges()
save(activities)
opt
[loadedCorruptData and file exists]
backupCorruptFile()
Copy the original bytes and verify
the .corrupt.bak file before writing.
getActivities()
insertion-order snapshot
Serialise the snapshot to nine-field rows.
Files.writeString(...) writes the UTF-8
contents to activities.txt.tmp.
moveIntoPlace(temporaryFile)
Attempt atomic replacement;
fall back on FileSystemException.
Clear recovery and unsaved-change flags after success.
If a save throws, keep unsavedChanges true and show a persistent warning.
Bye retries unsaved changes before exit without modifying activities.
A bye request is blocked until saving succeeds.
Display the command result through Ui
after any required save succeeds.


Validate an activity name or schedule edit before saving
ConsoleHelper
parser:Parser
activities:ActivityList
ActivityScheduleValidator
storage:Storage
ui:Ui
parse(input)
Reject non-ASCII command words before dispatch.
EditCommandParser(clock)
:EditCommandParser
parse(arguments)
Validate supplied syntax and field values.
ActivityNameValidator requires printable-ASCII replacement names.
Group supplied values in immutable Fields; null means omitted.
Fields(name, date, start, end, demand, importance, urgency, commitment)
changes:EditCommand.Fields
EditCommand(index, changes, clock, allowOverlap)
:EditCommand
editCommand
editCommand
execute(activities, storage)
getByDisplayedIndex(displayedIndex)
getCanonicalView()
existingActivity
LocalDateTime.now(clock)
This success-path example assumes the original end is after now
and any supplied date/time range ends after now.
Combine supplied fields with existing values.
Preserve completion and omitted ratings.
Activity(...)
replacement:Activity
Name/schedule edits run schedule validation.
Demand/rating-only edits skip it.
opt
[name or schedule field supplied]
validate(replacement, activities, existingActivity, allowOverlap)
getCanonicalView()
chronological activities
alt
[duplicate or unapproved time clash]
throws UniEnableException(warning)
propagate warning
showMessage(warning)
opt
[unsaved changes remain]
hasUnsavedChanges()
true
showMessage(persistent unsaved warning)
[validation passes or is skipped]
opt
[validation was requested]
validation passed
replaceByDisplayedIndex(displayedIndex, replacement)
getByDisplayedIndex(displayedIndex)
Replace the same chronological activity
at its backing-list insertion position.
result (didChangeActivities = true)
markUnsavedChanges()
save(activities)
alt
[save succeeds]
saved
showMessage(result.getMessage())
[save fails]
throws UniEnableException
showMessage(save error)
hasUnsavedChanges()
true
showMessage(persistent unsaved warning)
ConsoleHelper, not EditCommand, marks and saves the mutation.
Storage's serialization and recovery details are shown in CompletionAndSaveSequence.
The omitted rejection path for ended activities or invalid resulting times
throws before replacement and save. All rejected edits preserve activity data.


Find activities using chronological indexes
ConsoleHelper
parser:Parser
activities:ActivityList
ActivityListFormatter
ui:Ui
:FindCommand
parse(input)
Extract and trim the keyword.
Reject a blank keyword.
Unicode keywords remain allowed for saved names.
FindCommand(keyword)
findCommand
execute(activities, storage)
getCanonicalView()
canonicalView
findByName(keyword)
getCanonicalView()
Match name substrings using Locale.ROOT case folding.
Include completed and incomplete activities.
chronological matches
formatFindResults(keyword, matches, canonicalView)
Map each match to its full-list index + 1.
If there are no matches, build the no-match message.
message
result (didChangeActivities = false)
showMessage(result.getMessage())
The activity-change flag is false, so ConsoleHelper
does not call Storage.save(). Activity data is unchanged.


Mark an activity as completed
ConsoleHelper
parser:Parser
command:MarkCommand
activities:ActivityList
activity:Activity
storage:Storage
ui:Ui
The parser creates MarkCommand(1).
parse("mark 1")
command
execute(activities, storage)
getByDisplayedIndex(1)
activity
setDoneAtDisplayedIndex(1, true)
getByDisplayedIndex(1)
isDone()
current completion
alt [completion already equals true]
changed = false
[completion differs]
setDone(true)
changed = true
CommandResult(message, false, changed)
opt [result.didChangeActivities()]
markUnsavedChanges()
save(activities)
alt [save succeeds]
saved
[save fails]
throws UniEnableException
showMessage(save error)
showMessage(persistent unsaved warning)
opt [no save failure]
showMessage(result.getMessage())
hasUnsavedChanges()
status
opt [unsaved changes remain]
showMessage(persistent unsaved warning)
No change returns an already-in-that-state warning.
Invalid syntax or indexes are displayed as warnings without saving.
ConsoleHelper owns save handling; see CompletionAndSaveSequence.


Mark an activity as incomplete
ConsoleHelper
parser:Parser
command:UnmarkCommand
activities:ActivityList
activity:Activity
storage:Storage
ui:Ui
The parser creates UnmarkCommand(1).
parse("unmark 1")
command
execute(activities, storage)
getByDisplayedIndex(1)
activity
setDoneAtDisplayedIndex(1, false)
getByDisplayedIndex(1)
isDone()
current completion
alt [completion already equals false]
changed = false
[completion differs]
setDone(false)
changed = true
CommandResult(message, false, changed)
opt [result.didChangeActivities()]
markUnsavedChanges()
save(activities)
alt [save succeeds]
saved
[save fails]
throws UniEnableException
showMessage(save error)
showMessage(persistent unsaved warning)
opt [no save failure]
showMessage(result.getMessage())
hasUnsavedChanges()
status
opt [unsaved changes remain]
showMessage(persistent unsaved warning)
No change returns an already-in-that-state warning.
Invalid syntax or indexes are displayed as warnings without saving.
ConsoleHelper owns save handling; see CompletionAndSaveSequence.